

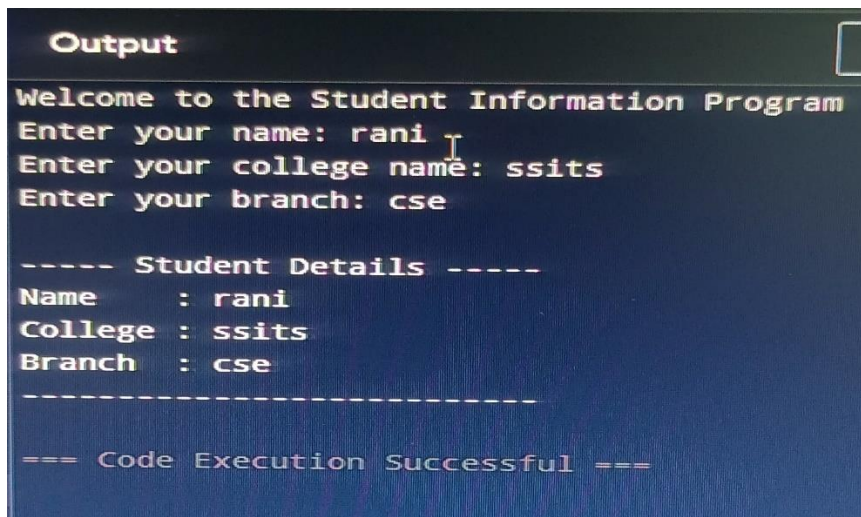
Assignment

Task-1: Python Basics

#Program

```
print("Welcome to the Student Information Program")
name = input("Enter your name: ")
college = input("Enter your college name: ")
branch = input("Enter your branch: ")
print("\n----- Student Details -----")
print("Name   :", name)
print("College :", college)
print("Branch  :", branch)
print("-----")
```

OUTPUT

A screenshot of a terminal window with a dark blue background and light green text. The window title is "Output". The text displayed is the output of a Python program: "Welcome to the Student Information Program", followed by three input prompts: "Enter your name: rani", "Enter your college name: ssits", and "Enter your branch: cse". Below these is a separator line "----- Student Details -----", followed by the formatted output: "Name : rani", "College : ssits", and "Branch : cse". Another separator line "-----" follows. At the bottom, it says "=== Code Execution Successful ===".

```
Output
Welcome to the Student Information Program
Enter your name: rani
Enter your college name: ssits
Enter your branch: cse

----- Student Details -----
Name      : rani
College   : ssits
Branch    : cse
-----

=== Code Execution Successful ===
```

Explanation

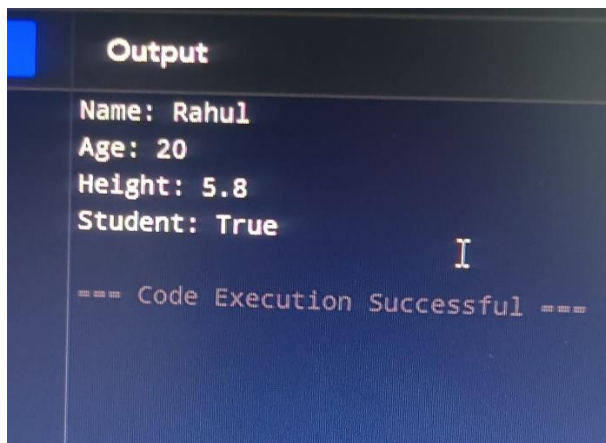
- The program starts by displaying a welcome message to the user.
- It takes the user's name, college name, and branch as input using the input() function.
- The entered values are stored in variables named name, college, and branch.
- Finally, the program prints the collected information in a neat and formatted output

Task-2 : Variables & Data Types

Program to demonstrate variables and data types

```
name = "Rahul"  
age = 20  
height = 5.8  
is_student = True  
print("Name:", name)  
print("Age:", age)  
print("Height:", height)  
print("Student:", is_student)
```

#OUTPUT:

A screenshot of a code editor's output window. The window has a dark blue background with a light blue header bar that says "Output". Below the header, the program's output is displayed in a monospaced font: "Name: Rahul", "Age: 20", "Height: 5.8", and "Student: True". At the bottom of the output, there is a line of text in a lighter color that says "=== Code Execution Successful ===". A white cursor is visible on the line containing "Student: True".

EXPLANATION

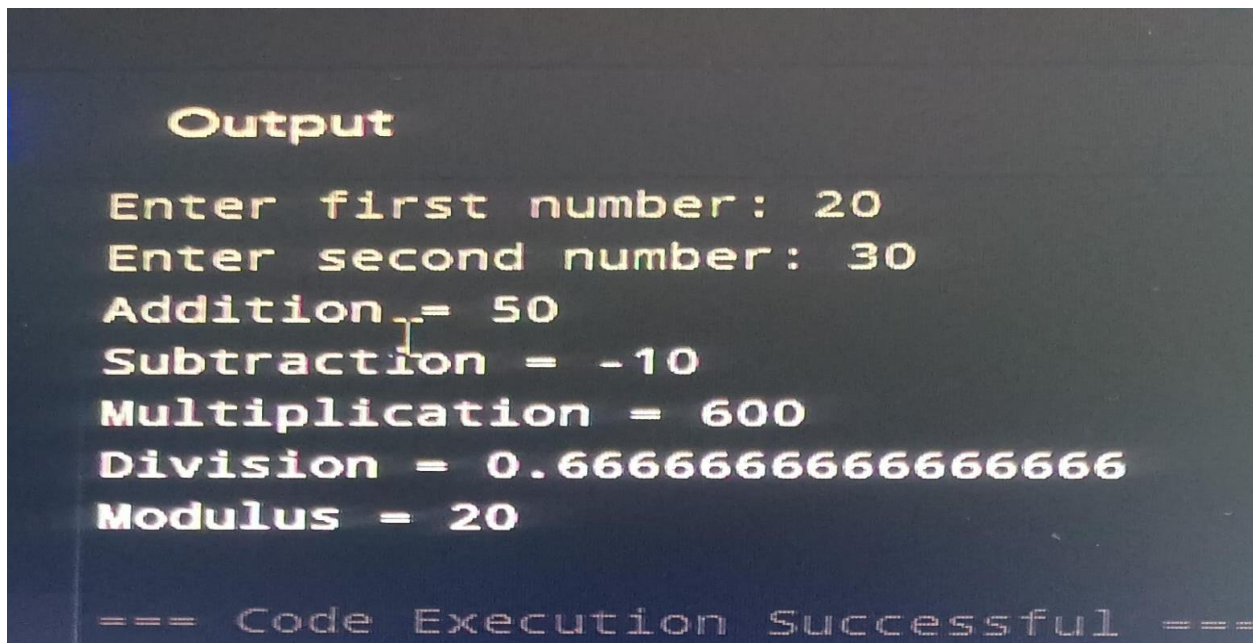
- The program demonstrates different types of variables in Python.
- It uses four data types: str (string), int (integer), float (decimal number), and bool (Boolean).
- Each value is stored in a variable with an appropriate data type.
- The print() function displays the values of all the variables on the screen.

#Task-3:Operators

program

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print("Addition =", a + b)
print("Subtraction =", a - b)
print("Multiplication =", a * b)
print("Division =", a / b)
print("Modulus =", a % b)
```

#OUTPUT:



```
Output
Enter first number: 20
Enter second number: 30
Addition = 50
Subtraction = -10
Multiplication = 600
Division = 0.6666666666666666
Modulus = 20

=== Code Execution Successful ===
```

Explanation:

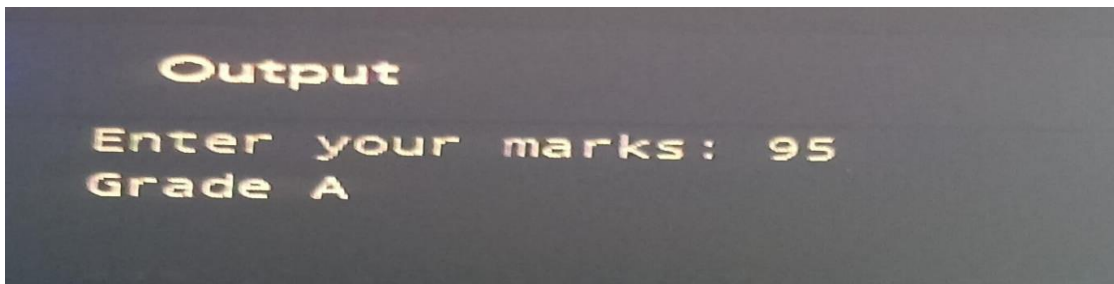
The + operator adds two numbers, - subtracts one number from another, * multiplies two numbers, and / performs division to give the quotient. The % operator returns the remainder after division. These arithmetic operators are used to perform basic mathematical calculations in Python. The program below takes two numbers from the user and displays the result of each operation.

#Task-4: Conditional Statements

program

```
marks = int(input("Enter your marks: "))  
if marks >= 90:  
    print("Grade A")  
elif marks >= 75:  
    print("Grade B")  
elif marks >= 60:  
    print("Grade C")  
else:  
    print("Fail")
```

#OUTPUT



Explanation

The if, elif, and else statements are decision-making statements in Python. They allow a program to execute different blocks of code based on whether a condition is True or False. The if statement checks the first condition, elif checks additional conditions if the previous ones are false, and else executes when none of the conditions are true. This is commonly used for grading systems, voting eligibility, and other situations where decisions depend on input values.

Program Explanation (4 lines)

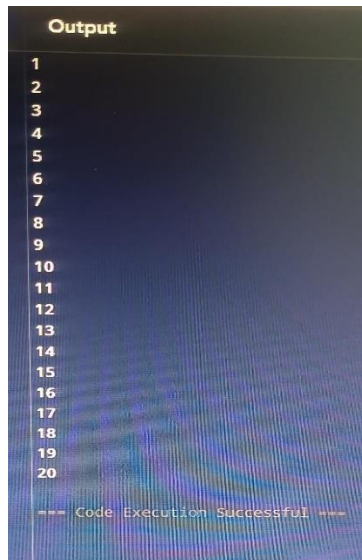
The program asks the user to enter their marks using the input() function. It checks the marks using if, elif, and else statements. If the marks are 90 or above, it displays Grade A; if they are 75–89, it displays Grade B; if they are 60–74, it displays Grade C; otherwise, it displays Fail.

#Task-5:Loops

Print numbers from 1 to 20

```
for i in range(1, 21):  
    print(i)
```

#OUTPUT



Explanation:

- range(1, 21) generates numbers from 1 to 20.
- The for loop iterates through each number.
- print(i) displays each number on a new line.

Multiplication table of any number

```
num = int(input("Enter a number: "))  
for i in range(1, 11):  
    print(num, "x", i, "=", num * i)
```

#OUTPUT

```
Output
Enter a number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Explanation

- Takes a number as input from the user.
- Uses a for loop (range(1, 11)) to print the multiplication table from 1 to 10.

Print even numbers from 1 to 50 using a while loop

```
num = 2
```

```
while num <= 50:
    print(num)
    num = num + 2
```

#OUTPUT:

```
Output
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
40
42
44
46
48
50
=== Code Execution Successful ===
```


#Explanation

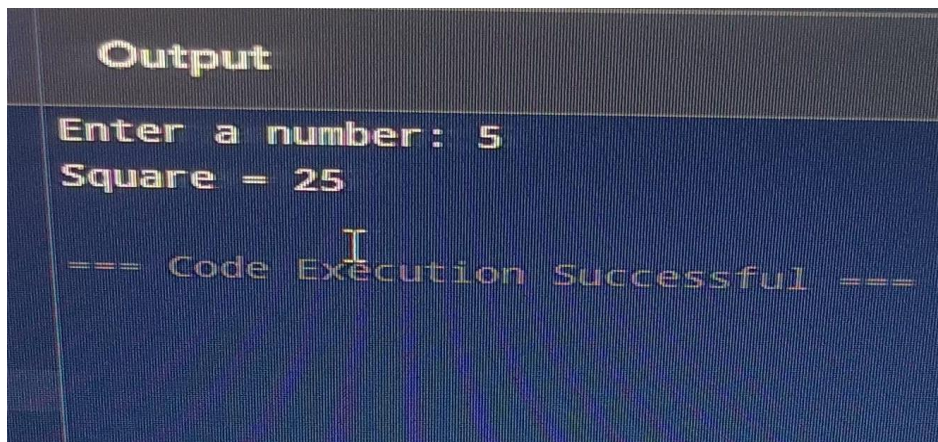
- Starts with num = 2 (the first even number).
- Uses a while loop to print numbers up to 50.
- Increases num by 2 each time, so only even numbers are printed.

#Task-6: Functions

Function to calculate the square of a number

```
def square(num):  
    return num * num  
  
number = int(input("Enter a number: "))  
print("Square =", square(number))
```

OUTPUT



```
Output  
Enter a number: 5  
Square = 25  
=== Code Execution Successful ===
```

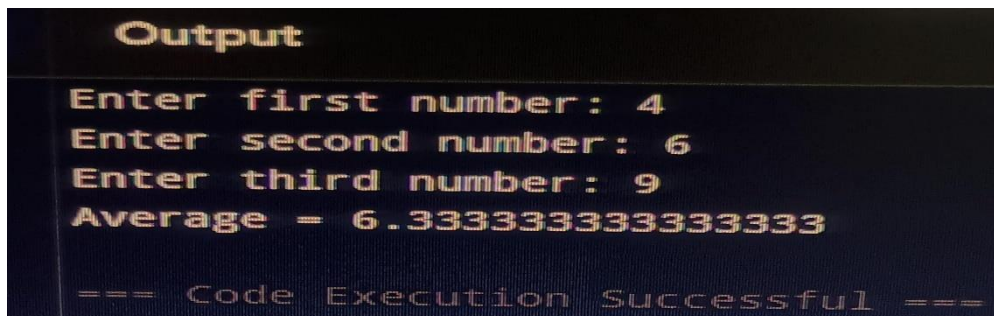
Explanation:

- def square(num): defines a function named square.
- return num * num returns the square of the number.
- The user enters a number.
- The function is called, and the result is displayed.

Function to calculate the average of three numbers

```
def average(a, b, c):  
    return (a + b + c) / 3  
  
num1 = float(input("Enter first number: "))  
num2 = float(input("Enter second number: "))  
num3 = float(input("Enter third number: "))  
print("Average =", average(num1, num2, num3))
```

#OUTPUT



```
Output  
Enter first number: 4  
Enter second number: 6  
Enter third number: 9  
Average = 6.333333333333333  
  
=== Code Execution Successful ===
```

Explanation:

- `def average(a, b, c):` defines a function named `average`.
- `(a + b + c) / 3` calculates the average of the three numbers.
- The user enters three numbers.
- The function returns the average, which is then printed.

#Task-7:Strings & Collections

String operations

```
text = "Hello Python"  
  
print("Original String:", text)  
print("Upper Case:", text.upper())  
print("Lower Case:", text.lower())  
print("Replace:", text.replace("Python", "World"))  
print("Find 'Python':", text.find("Python"))
```


#OUTPUT:

```
Output
Original String: Hello Python
Upper Case: HELLO PYTHON
Lower Case: hello python
Replace: Hello World
Find 'Python': 6

=== Code Execution Successful ===
```

Explanation

- upper() converts the string to uppercase.
- lower() converts the string to lowercase.
- replace() replaces one word with another.
- find() returns the position of a word in the string.

List operations

```
numbers = [40, 10, 30, 20]
numbers.append(50)
print("After append:", numbers)
numbers.remove(30)
print("After remove:", numbers)
numbers.sort()
print("After sort:", numbers)
```

#OUTPUT

```
Output
After append: [40, 10, 30, 20, 50]
After remove: [40, 10, 20, 50]
After sort: [10, 20, 40, 50]

=== Code Execution Successful ===
```

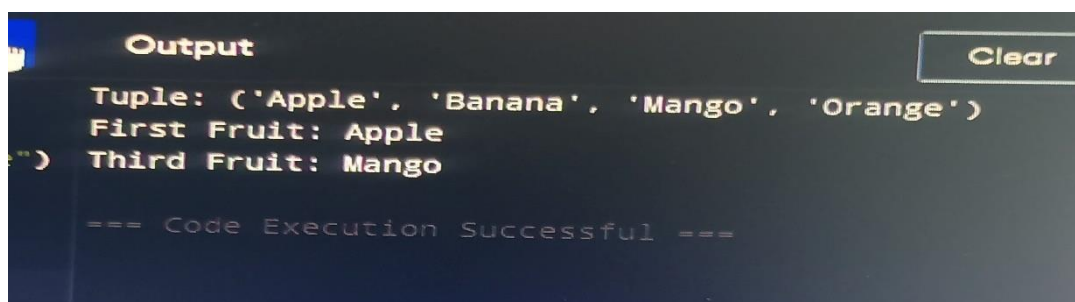
Explanation:

- `append()` adds an element to the end of the list.
- `remove()` deletes the specified element.
- `sort()` arranges the list in ascending order.

Tuple creation and indexing

```
fruits = ("Apple", "Banana", "Mango", "Orange")  
print("Tuple:", fruits)  
print("First Fruit:", fruits[0])  
print("Third Fruit:", fruits[2])
```

#OUTPUT:

A screenshot of a code execution environment. At the top, there's a header 'Output' with a 'Clear' button on the right. Below the header, the output of the code is displayed in a monospaced font: 'Tuple: ('Apple', 'Banana', 'Mango', 'Orange')', 'First Fruit: Apple', and 'Third Fruit: Mango'. At the bottom, a status message reads '=== Code Execution Successful ==='.

```
Output Clear  
Tuple: ('Apple', 'Banana', 'Mango', 'Orange')  
First Fruit: Apple  
Third Fruit: Mango  
=== Code Execution Successful ===
```

Explanation

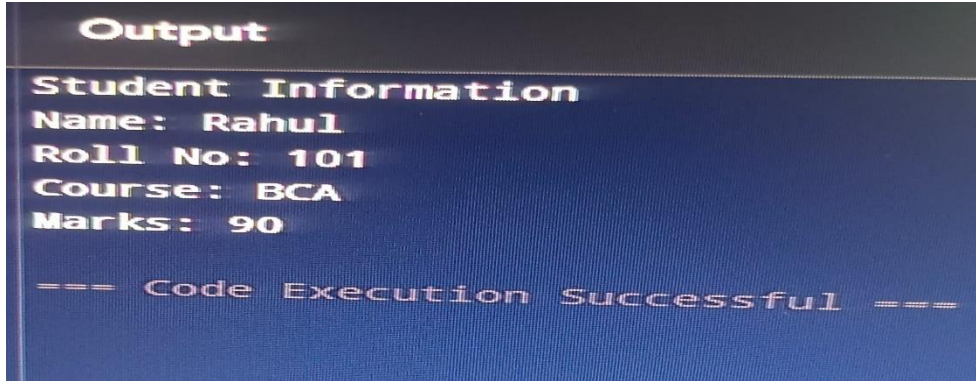
- A tuple is created using parentheses ().
- Tuple elements are accessed using their index.
- Tuples cannot be modified after creation.

Dictionary example

```
student = {  
    "Name": "Rahul",  
    "Roll No": 101,  
    "Course": "BCA",  
    "Marks": 90  
}  
  
print("Student Information")  
print("Name:", student["Name"])
```

```
print("Roll No:", student["Roll No"])
print("Course:", student["Course"])
print("Marks:", student["Marks"])
```

#OUTPUT:



```
Output
Student Information
Name: Rahul
Roll No: 101
Course: BCA
Marks: 90

--- Code Execution Successful ---
```

Explantion:

- A dictionary stores data in key-value pairs.
- It is created using curly braces {}.
- Each key is unique and is used to access its corresponding value.
- In this program:
- "Name", "Roll No", "Course", and "Marks" are keys.
"Rahul", 101, "BCA", and 90 are the values.
- student["Name"] displays the value stored for the key "Name".

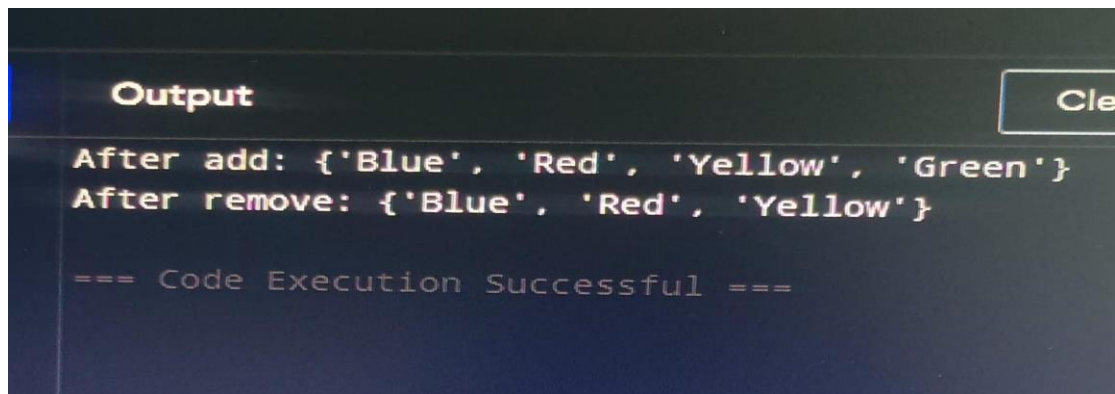
Set operations

```
colors = {"Red", "Green", "Blue"}
```

```
colors.add("Yellow")
print("After add:", colors)
```

```
colors.remove("Green")
print("After remove:", colors)
```

#OUTPUT:

A screenshot of a code execution output window. The window has a dark background with a 'Output' label in the top left and a 'Clear' button in the top right. The output text is as follows:

```
After add: {'Blue', 'Red', 'Yellow', 'Green'}  
After remove: {'Blue', 'Red', 'Yellow'}  
  
=== Code Execution Successful ===
```

Explanation:

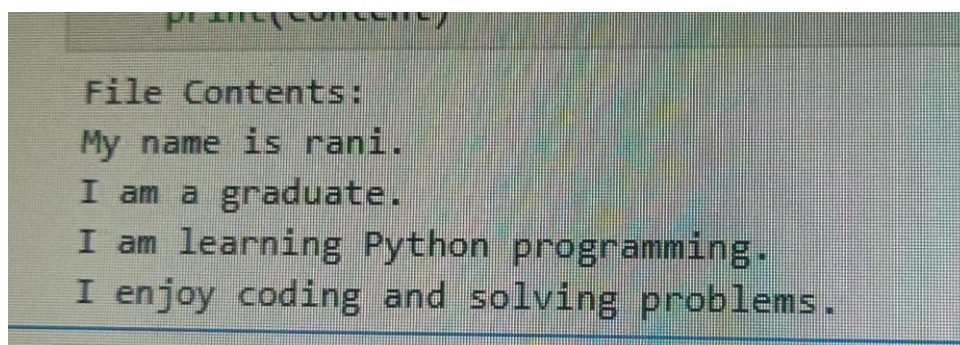
- add() inserts a new element into the set.
- remove() deletes the specified element.
- A set stores unique elements only.

#Task-8:File Handling

```
# Create a file and write an introduction  
with open("introduction.txt", "w") as file:  
    file.write("My name is rani.\n")  
    file.write("I am a graduate.\n")  
    file.write("I am learning Python programming.\n")  
    file.write("I enjoy coding and solving problems.")
```

```
# Read and display the file contents  
with open("introduction.txt", "r") as file:  
    content = file.read()  
    print("File Contents:")  
    print(content)
```

#OUTPUT

A screenshot of a code execution output window. The window has a light background. The output text is as follows:

```
File Contents:  
My name is rani.  
I am a graduate.  
I am learning Python programming.  
I enjoy coding and solving problems.
```

Explanation:

- This program demonstrates basic file handling operations in Python.
- It creates a text file named introduction.txt and writes introduction details into it.
- The program then opens the same file in read mode and reads all the contents. Finally, it displays the file contents on the screen using the print() function.

#Task-9:Mini Python Project

#Develop a Student Record Management System using Python.

```
# Student Record Management System
```

```
students = []
```

```
def add_student():
```

```
    name = input("Enter student name: ")
```

```
    roll = int(input("Enter roll number: "))
```

```
    course = input("Enter course: ")
```

```
    student = {
```

```
        "Name": name,
```

```
        "Roll No": roll,
```

```
        "Course": course
```

```
    }
```

```
    students.append(student)
```

```
    print("Student added successfully!\n")
```

```
def display_students():
```

```
    if len(students) == 0:
```

```
        print("No student records found.\n")
```

```
    else:
```

```
        print("\nStudent Records:")
```

```
        for student in students:
```

```
            print("Name:", student["Name"])
```

```
            print("Roll No:", student["Roll No"])
```

```
            print("Course:", student["Course"])
```

```
            print("-----")
```

```

def search_student():
    name = input("Enter student name to search: ")

    found = False
    for student in students:
        if student["Name"].lower() == name.lower():
            print("Student Found:")
            print("Name:", student["Name"])
            print("Roll No:", student["Roll No"])
            print("Course:", student["Course"])
            found = True
            break

    if not found:
        print("Student not found.\n")

def delete_student():
    name = input("Enter student name to delete: ")

    for student in students:
        if student["Name"].lower() == name.lower():
            students.remove(student)
            print("Student record deleted successfully!\n")
            return

    print("Student not found.\n")

while True:
    print("\n--- Student Record Management System ---")
    print("1. Add Student")
    print("2. Display Students")
    print("3. Search Student")
    print("4. Delete Student")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        add_student()

    elif choice == 2:
        display_students()

    elif choice == 3:

```



```

        search_student()

    elif choice == 4:
        delete_student()

    elif choice == 5:
        print("Program exited.")
        break

    else:
        print("Invalid choice. Please try again.")

```

#OUTPUT

```

Output

--- Student Record Management System ---
1. Add Student
2. Display Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice: 1
Enter student name: rahul
Enter roll number: 591
Enter course: cse
Student added successfully!

--- Student Record Management System ---
1. Add Student
2. Display Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice: 2

Student Records:      I
Name: rahul
Roll No: 591
Course: cse
-----

--- Student Record Management System ---
1. Add Student
2. Display Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice: |

```

Explanation:

- A list named students is used to store all student records.
- Each student record is stored as a dictionary containing:
- Student Name

- Roll Number
- Course
- The program uses functions for different operations:
- `add_student()` → Adds new student details.
- `display_students()` → Shows all saved student records.
- `search_student()` → Finds a student by name.
- `delete_student()` → Removes a student's record.
- A menu-driven loop allows the user to select different operations until they choose Exit.